Dependency Injection

1. What is Dependency Injection?

Dependency Injection (DI) is a design pattern where a class receives the objects it depends on from the outside, rather than creating them internally. In simple terms:

> “Don't build your own dependencies — have them provided to you.”

2. Why do we use Dependency Injection? (Core reasons)

1. Reduces coupling

The class does not control how dependencies are created. Code becomes more modular and flexible.

2. Easier unit testing

You can inject mock objects instead of real implementations.

3. Improved maintainability

Changing a dependency implementation requires no change to the consuming class.

4. Supports SOLID principles

> Especially D — Dependency Inversion Principle.

3. Types of DI

Constructor Injection

- Most preferred
- Immutable
- Best for testing

Setter Injection

- For optional dependencies.

Field Injection

- Not recommended (hurts testability) Still seen in legacy code.

5. Common Interview Questions & Short Answers

Q1: Why is constructor injection preferred?

- Ensures dependencies are never null
- Supports immutability
- Works best for unit tests

Q2: What is Inversion of Control (IoC) and how does DI relate?

- IoC = external entity controls object creation
- DI = one form of IoC
- Spring IoC container = dependency injector

Q3: What problem does DI solve?

- Tight coupling
- Hard-coded dependencies
- Difficult testing

Q4: How does Spring decide which implementation to inject?

By type, then by qualifiers Uses @Primary or @Qualifier

Design Patterns

What Are Design Patterns?

Design patterns are reusable solutions to common software design problems. They are not code templates

> They are best-practice ways to structure your code so it is flexible, maintainable, and scalable.

Patterns help you avoid:

- duplicated logic
- tight coupling
- brittle architecture

Categories of Design Patterns

- Creational Patterns → object creation
- Structural Patterns → object composition
- Behavioral Patterns → interactions & algorithms

Most Important Patterns for Java Interviews

CREATIONAL PATTERNS

1. Singleton Pattern

Ensures only one instance of a class exists.

```
public class Config {
    private static final Config INSTANCE = new Config();
    private Config() {}
    public static Config getInstance() {
        return INSTANCE;
```

```
    }
}
```

> Used for:

```
* configuration
* caching
* thread pools
```

💡 Spring beans are singleton scoped by default.

2. Factory Method Pattern

Creates objects without exposing creation logic.

```
interface Transport { void deliver(); }

class Truck implements Transport { public void deliver() {} }
class Ship implements Transport { public void deliver() {} }

class TransportFactory {
    public static Transport create(String type) {
        return switch(type) {
            case "truck" -> new Truck();
            case "ship" -> new Ship();
            default -> throw new IllegalArgumentException();
        };
    }
}
```

☑ Used when selecting implementation at runtime

3. Builder Pattern

Simplifies building complex objects.

```
User user = new User.Builder()
        .name("Leonardo")
        .age(30)
        .email("leo@test.com")
        .build();
```

☑ Seen in:

- StringBuilder

- Lombok @Builder
- Spring HTTP builders

Structural Patterns

4. Adapter Pattern

Allows incompatible interfaces to work together.

```
class SquarePegAdapter implements RoundPeg {
    private SquarePeg peg;

    public SquarePegAdapter(SquarePeg peg) {
        this.peg = peg;
    }

    @Override
    public double getRadius() {
        return peg.getWidth() * Math.sqrt(2);
    }
}
```

☑ Used when integrating legacy code

5. Decorator Pattern

Adds behavior dynamically.

```
class LoggingService implements DataService {
    private final DataService service;

    LoggingService(DataService service) {
        this.service = service;
    }

    public void save() {
        System.out.println("Logging...");
        service.save();
    }
}
```

☑ Used massively in Spring with:

@Transactional @Cacheable Web filters

Behavioral Patterns

6. Strategy Pattern

Switch behavior at runtime.

```
interface PaymentStrategy { void pay(); }

class PayPal implements PaymentStrategy { public void pay() {} }
class CreditCard implements PaymentStrategy { public void pay() {} }

class Checkout {
    private PaymentStrategy method;
    Checkout(PaymentStrategy method) { this.method = method; }
    public void process() { method.pay(); }
}
```

☑ Used for:

payment methods sorting algorithms compression strategies

7. Observer Pattern

Notifies subscribers of state changes.

```
interface Observer { void update(); }

class EmailObserver implements Observer { public void update() {} }

class Order {
    private List<Observer> observers = new ArrayList<>();
    public void add(Observer o) { observers.add(o); }
    public void complete() { observers.forEach(Observer::update); }
}
```

☑ Used in:

event systems UI frameworks message listeners

8. Command Pattern

Encapsulates an operation as an object.

```
interface Command { void execute(); }

class SaveCommand implements Command {
    public void execute() { System.out.println("Saving..."); }
}
```

☑ Used for:

undo/redo job executors task queues

Patterns Common in Spring Boot

You should highlight these in an interview:

- @Autowired -> Dependency Injection
- ApplicationContext -> Factory + Singleton
- HandlerMapping -> Strategy
- AOP (@Transactional) -> Proxy / Decorator
- Events -> Observer

What patterns do you use?

> I frequently use Singleton and Factory patterns for bean creation, Strategy for algorithm selection, Builder for constructing complex objects, and Decorator/Proxy in Spring via AOP annotations like @Transactional. I also rely on Observer for event-driven features.

equals() and hashCode() methods

What is the purpose of the equals() method? When would you override it?

equals() checks logical equality between objects (e.g., two Users with the same ID should be considered the same). You override it when:

identity is not enough objects represent logical entities (User, Product, Book) you want proper comparison in collections (HashSet, List.contains())

What is the purpose of hashCode()? Why do hash collections need it?

hashCode() returns an integer used to place objects into buckets in hash-based collections. Collections that need it:

- HashMap
- HashSet
- LinkedHashMap
- ConcurrentHashMap

It improves lookup speed by narrowing the search to a specific bucket.

Explain the contract between equals() and hashCode().

Contract:

If two objects are equal via equals(), they must have the same hash code. If two objects have the same hash code, they are not necessarily equal (collisions allowed). If you override equals(), you must also override hashCode().

Breaking this contract results in objects disappearing from hash collections or failing to be found.

JPA/Hibernate pitfalls with overriding equals() & hashCode().

Common issues:

Using database-generated IDs in equals() leads to problems before persistence. Objects may not be equal until after being saved. Recommended: use natural keys (email, username) or immutable business keys.

How to design equals() and hashCode() for:

- Immutable class

> Safe to use all fields.

- Mutable class

> Only use immutable identity fields (e.g., id). Never use fields that may change after insertion into hash collections.

What's wrong with this implementation?

```
public boolean equals(User other)
```

Issues:

- Does not override Object.equals(Object) → method overloading.
- Never called by collections.
- No null/type check.
- Missing hashCode().

What bugs exist here?

```
@Override
public boolean equals(Object o) {
    User other = (User) o;
    return this.id == other.id;
}
```

Problems:

- No null check → NullPointerException.
- No instanceof check → ClassCastException.
- Breaks symmetry if other type occurs.

Does this respect the contract?

```
@Override
public boolean equals(Object o) { return true; }
@Override
public int hashCode() { return 1; }
```

Technically: yes. Practically: terrible. -> All objects end up in the same bucket → O(N) lookup times → horrible performance.

What's wrong with this class?

```
class Point { int x, y; }
```

- Uses default equals() and hashCode() (identity comparison).
- So new Point(1,2) and new Point(1,2) are considered different.
- In a HashSet, both are inserted → duplicates.

> Fix: override both methods.

Why prefer Objects.equals() and Objects.hash()?

- null-safe
- concise
- consistent
- avoids manual mistakes
- widely accepted modern Java standard

Collections

What are Collections?

Collections are containers that hold groups of objects. They provide efficient ways to store, retrieve, and manipulate data.

Collection Hierarchy

```
Iterable
  ↓
Collection
  ├── List (ordered, allows duplicates)
  ├── Set (unique values, unordered)
  └── Queue (FIFO/priority)

Map (key-value pairs, not a Collection)
```

Common Collections

Type	Implementation	Use Case	Thread-Safe
List	ArrayList, LinkedList	Ordered data, indexed access	Collections.synchronizedList()
Set	HashSet, TreeSet, LinkedHashSet	Unique values, fast lookups	Collections.synchronizedSet()
Map	HashMap, TreeMap, LinkedHashMap	Key-value pairs	Collections.synchronizedMap()
Queue	LinkedList, PriorityQueue	FIFO, priority-based	ConcurrentLinkedQueue

ArrayList vs LinkedList

Feature	ArrayList	LinkedList
Access	O(1)	O(n)
Insert/Delete	O(n) - shifts elements	O(1) - pointer updates
Memory	Contiguous	Non-contiguous
Use When	Frequent reads	Frequent inserts/deletes

Bad Example:

```
LinkedList<Integer> list = new LinkedList<>();
for (int i = 0; i < 1000; i++) {
    System.out.println(list.get(i)); // O(n) for each access = O(n²)
}
```

Good Example:

```
ArrayList<Integer> list = new ArrayList<>();
for (int i = 0; i < 1000; i++) {
    System.out.println(list.get(i)); // O(1) for each access = O(n)
}
```

HashMap vs TreeMap vs LinkedHashMap

HashMap:

- O(1) average lookup
- No ordering
- Best for general key-value storage

TreeMap:

- O(log n) lookup
- Sorted by key
- Use when you need order or range queries

LinkedHashMap:

- O(1) lookup + insertion order preserved
- Useful for LRU caches

Example:

```
Map<String, Integer> map = new HashMap<>();
map.put("apple", 5);
map.put("banana", 3);

// Iteration order is unpredictable
for (String key : map.keySet()) {
    System.out.println(key); // random order
}

// Use LinkedHashMap to preserve insertion order
Map<String, Integer> ordered = new LinkedHashMap<>();
ordered.put("apple", 5);
ordered.put("banana", 3);

for (String key : ordered.keySet()) {
    System.out.println(key); // apple, banana
}
```

HashSet vs TreeSet

HashSet:

- O(1) lookup, add, remove
- Unordered
- Allows null values

TreeSet:

- O(log n) operations
- Sorted (natural or custom comparator)
- No nulls allowed

Example:

```
Set<Integer> set = new HashSet<>(Arrays.asList(5, 2, 8, 2));
System.out.println(set); // [2, 5, 8] - no duplicates, unordered
```

```
Set<Integer> sorted = new TreeSet<>(Arrays.asList(5, 2, 8, 2));
System.out.println(sorted); // [2, 5, 8] - sorted
```

Interview Questions on Collections

Q: Why does HashMap allow null but TreeMap doesn't?

> TreeMap needs to compare keys for ordering. Nulls can't be compared. HashMap just uses hash codes, so nulls are allowed.

Q: When would you use LinkedHashMap over HashMap?

> When you need insertion order preserved, like implementing an LRU cache.

Q: What's the difference between fail-fast and fail-safe iterators?

> Fail-fast throws ConcurrentModificationException if collection is modified during iteration (ArrayList). Fail-safe works on a copy (CopyOnWriteArrayList).

String Handling

String Immutability

Strings in Java are immutable. Once created, they cannot be changed.

```
String s1 = "Hello";
String s2 = s1.concat(" World"); // Creates a new String
System.out.println(s1); // "Hello" - unchanged
System.out.println(s2); // "Hello World"
```

Why Immutable?

- Thread-safe
- Safe as HashMap keys
- Secure (passwords)
- Can be cached

String Pool

Java maintains a String pool for memory efficiency. String literals are cached.

```
String s1 = "Hello"; // String pool
String s2 = "Hello"; // Retrieves from pool
```

```
System.out.println(s1 == s2); // true - same reference

String s3 = new String("Hello"); // Creates new object
System.out.println(s1 == s3); // false - different references
System.out.println(s1.equals(s3)); // true - same value
```

☑ Always use `.equals()` for String comparison, not `==`.

StringBuilder vs StringBuffer vs String Concatenation

Approach	Thread-Safe	Performance	Use When
String	Yes	Slow (creates new objects)	Single concatenation
StringBuilder	No	Fast	Loop concatenations
StringBuffer	Yes	Medium	Multi-threaded environments

Bad Example:

```
String result = "";
for (int i = 0; i < 1000; i++) {
    result += "Item" + i; // Creates 1000 new String objects
}
```

Good Example:

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) {
    sb.append("Item").append(i);
}
String result = sb.toString(); // Single object
```

String Methods - Common Interview Questions

```
String s = "Hello World";

s.length();           // 11
s.charAt(0);          // 'H'
s.substring(0, 5);    // "Hello"
s.indexOf("World");   // 6
s.contains("World");  // true
s.toUpperCase();      // "HELLO WORLD"
s.split(" ");         // ["Hello", "World"]
```

```
s.trim();              // Removes leading/trailing spaces
s.replace("World", "Java"); // "Hello Java"
```

Interview Questions on Strings

Q: Why use StringBuilder in loops?

> String concatenation creates a new object each time. StringBuilder appends to a mutable buffer, much faster for repeated operations.

Q: What's the difference between `==` and `.equals()`?

> `==` checks reference equality (same object in memory). `.equals()` checks value equality (same content).

Q: Can you use String as a HashMap key?

> Yes, because strings are immutable and have consistent hash codes.

Streams & Functional Programming

What are Streams?

Streams (Java 8+) are a functional way to process data collections. They enable functional programming with lazy evaluation and method chaining.

Stream vs Collections

- **Collections:** eager evaluation, store data in memory
- **Streams:** lazy evaluation, process-on-demand, don't store data

---

Common Stream Operations

Intermediate Operations (return Stream)

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

numbers.stream()
    .filter(n -> n > 2)           // Keep only n > 2
    .map(n -> n * 2)              // Transform: multiply by 2
    .distinct()                   // Remove duplicates
    .sorted()                      // Sort
    .limit(2)                      // Take first 2
    .forEach(System.out::println); // Output: 6, 8
```

Terminal Operations (execute the stream)

```
// Collect: gather results
List<Integer> result = stream.collect(Collectors.toList());

// ForEach: iterate
stream.forEach(System.out::println);

// Reduce: combine elements
int sum = stream.reduce(0, Integer::sum);

// Count: count elements
long count = stream.count();

// FindFirst: get first element
Optional<Integer> first = stream.findFirst();

// AnyMatch: check if any match condition
boolean hasEven = stream.anyMatch(n -> n % 2 == 0);
```

Real-world Stream Examples

Example 1: Filter and Count

```
List<User> users = getUsers();

long activeAdminCount = users.stream()
    .filter(User::isActive)
    .filter(u -> u.getRole().equals("ADMIN"))
    .count();
```

Example 2: Transform and Collect

```
List<Order> orders = getOrders();

List<String> customerEmails = orders.stream()
    .map(Order::getCustomer)
    .map(Customer::getEmail)
    .distinct()
    .collect(Collectors.toList());
```

Example 3: Reduce

```
List<Integer> prices = Arrays.asList(10, 20, 30, 40);
```

```
int totalPrice = prices.stream()
    .reduce(0, Integer::sum); // Result: 100
```

Example 4: GroupingBy

```
List<Employee> employees = getEmployees();

Map<String, List<Employee>> byDepartment = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));
```

Lazy Evaluation

Streams don't execute until a terminal operation is called.

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

Stream<Integer> stream = numbers.stream()
    .filter(n -> {
        System.out.println("Filtering: " + n);
        return n > 2;
    })
    .map(n -> {
        System.out.println("Mapping: " + n);
        return n * 2;
    });

// Nothing printed yet - lazy!

stream.forEach(System.out::println); // NOW it executes
```

☑ Use streams for complex data transformations. Use loops for simple iterations.

Interview Questions on Streams

Q: What's the difference between map() and flatMap()?

> `map()` applies a function to each element. `flatMap()` applies a function that returns a Stream and flattens the result.

```
List<List<Integer>> lists = Arrays.asList(
    Arrays.asList(1, 2),
    Arrays.asList(3, 4)
);

// map returns Stream<List<Integer>>
```

```
lists.stream().map(List::stream);

// flatMap returns Stream<Integer>
lists.stream().flatMap(List::stream).forEach(System.out::println); // 1, 2, 3, 4
```

Q: When should you use streams vs loops?

> Use streams for complex transformations (filter, map, reduce). Use loops for simple iterations or when you need to break early.

Q: Are streams reusable?

> No. Once consumed, a stream is closed. You must create a new stream.

```
Stream<Integer> stream = numbers.stream().filter(n -> n > 2);
stream.forEach(System.out::println);
stream.forEach(System.out::println); // ❌ IllegalStateException
```

Concurrency

Threads vs Processes

- **Process:** Independent program with its own memory
- **Thread:** Lightweight, shares memory with other threads in same process

---

Creating Threads

Option 1: Extend Thread class

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running");
    }
}

MyThread thread = new MyThread();
thread.start(); // Calls run() in a new thread
```

Option 2: Implement Runnable (preferred)

```
class MyTask implements Runnable {
    public void run() {
        System.out.println("Task running");
    }
```

```
}

Thread thread = new Thread(new MyTask());
thread.start();

// Or with lambda
new Thread(() -> System.out.println("Task running")).start();
```

☑ Prefer Runnable because Java doesn't support multiple inheritance.

Thread Lifecycle

```
NEW → RUNNABLE → RUNNING → BLOCKED/WAITING → TERMINATED
```

Synchronization

The Problem: Race Condition

```
class Counter {
    private int count = 0;

    public void increment() {
        count++; // ❌ Not atomic - three steps: read, increment, write
    }
}

// If two threads call increment() simultaneously, count might only increase by 1
```

Solution 1: Synchronized Methods

```
class Counter {
    private int count = 0;

    public synchronized void increment() { // Only one thread at a time
        count++;
    }
}
```

Solution 2: Synchronized Blocks

```
class Counter {
    private int count = 0;
```

```
    public void increment() {
        synchronized(this) { // Lock this object
            count++;
        }
    }
}
```

Solution 3: AtomicInteger (Preferred for simple operations)

```
import java.util.concurrent.atomic.AtomicInteger;

class Counter {
    private AtomicInteger count = new AtomicInteger(0);

    public void increment() {
        count.incrementAndGet(); // Thread-safe, no locks needed
    }
}
```

Deadlock

Occurs when two threads wait for each other's locks indefinitely.

```
// ❌ Risk of deadlock
class Account {
    synchronized void transfer(Account other, int amount) {
        this.balance -= amount;
        other.balance += amount; // If other is locked, deadlock!
    }
}

// Account A calls transfer to B
// Account B calls transfer to A
// Both wait for each other → deadlock
```

Solution: Always acquire locks in the same order.

Thread Pool (ExecutorService)

Instead of creating threads manually, use a thread pool.

```
import java.util.concurrent.*;

ExecutorService executor = Executors.newFixedThreadPool(5);
```

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

numbers.forEach(n -> {
    executor.submit(() -> {
        System.out.println("Processing: " + n);
    });
});

executor.shutdown(); // No new tasks accepted
executor.awaitTermination(1, TimeUnit.MINUTES); // Wait for completion
```

Future and Callable

When you need a return value from a task:

```
ExecutorService executor = Executors.newFixedThreadPool(2);

// Callable returns a value
Callable<Integer> task = () -> {
    return 5 * 5; // Expensive computation
};

// Future represents the result
Future<Integer> result = executor.submit(task);

try {
    Integer value = result.get(); // Blocks until result is ready
    System.out.println("Result: " + value); // 25
} catch (InterruptedException | ExecutionException e) {
    e.printStackTrace();
}

executor.shutdown();
```

Interview Questions on Concurrency

Q: What's the difference between synchronized and volatile?

> `synchronized` ensures only one thread at a time, allows proper memory visibility. `volatile` ensures memory visibility but doesn't provide atomicity.

Q: What causes a deadlock?

> Circular wait: Thread 1 holds Lock A and waits for Lock B. Thread 2 holds Lock B and waits for Lock A.

Q: When would you use AtomicInteger over synchronized?

> For simple operations (increment, set). Atomic classes are faster and cleaner. Use synchronized for complex multi-step operations.

Q: What's the difference between wait() and sleep()?

> `sleep()` pauses a thread for a duration. `wait()` releases the lock and waits for a notify() call.

```
// sleep doesn't release lock
synchronized void method() {
    Thread.sleep(1000); // Holds lock for 1 second
}

// wait releases lock and waits for notify
synchronized void method() {
    wait(); // Releases lock, waits for notify()
}
```

Q: What's the benefit of using a thread pool?

> Reusing threads is cheaper than creating/destroying them. Pool manages thread lifecycle and prevents resource exhaustion from creating too many threads.

Q: Have you dealt with concurrency issues?

> Yes. I've used synchronized blocks for shared state, atomic classes for counters, and ExecutorService for parallel processing. I'm careful about deadlocks and always acquire locks in a consistent order.

Clean Code

What is Clean Code?

Clean code is code that is easy to read, understand, modify, and maintain. It's not just about making software work—it's about making it work in a way that minimizes the time and mental effort needed by anyone reading it.

> **Robert C. Martin (Uncle Bob):** "Clean code always looks like it was written by someone who cares."

Key Principles of Clean Code

1. Meaningful Names

Use clear, intention-revealing names that reflect purpose.

Bad:

```
int d; // days? data? duration?
String s; // what does this string contain?
public void process() {} // what is being processed?
```

Good:

```
int daysSinceCreation;
String customerEmail;
public void sendNotificationEmail() {}
```

☑ Names should answer: **Why does it exist? What does it do? How is it used?**

2. Functions Should Be Small and Focused

Aim for single responsibility.

Bad:

```
public void processOrderAndSendEmail(Order order) {
    // validate order
    // calculate total
    // apply discount
    // save to database
    // send confirmation email
    // send notification to inventory
}
```

Good:

```
public void processOrder(Order order) {
    validateOrder(order);
    calculateTotal(order);
    saveOrder(order);
    notifyEmail(order);
}
```

☑ **One function = One reason to change**

3. Comments Should Not Compensate for Bad Code

Good code shouldn't need lengthy explanations.

Bad:

```
// Check if the user has permissions and is not disabled
if (user.getRole().equals("ADMIN") && user.isActive()) {
    // Allow access
}
```

Good:

```
if (user.isAdmin() && user.isActive()) {
    allowAccess();
}
```

☑ Only use comments to explain **why**, not **what**. The code itself should explain **what**.

4. Error Handling Should Be Clean

Don't ignore exceptions; handle them properly.

Bad:

```
try {
    readFile();
} catch (Exception e) {
    e.printStackTrace(); // Swallowing exceptions
}
```

Good:

```
try {
    readFile();
} catch (IOException e) {
    log.error("Failed to read configuration file", e);
    throw new ConfigurationException("Unable to load config", e);
}
```

5. Keep It DRY (Don't Repeat Yourself)

Eliminate code duplication.

Bad:

```
public void validateEmail(String email) {
    if (!email.contains("@")) throw new Exception("Invalid email");
}

public void validateUserEmail(String email) {
    if (!email.contains("@")) throw new Exception("Invalid email");
}
```

Good:

```
private boolean isValidEmail(String email) {
    return email.contains("@") && email.length() > 5;
}

public void validateEmail(String email) {
    if (!isValidEmail(email)) throw new InvalidEmailException();
}
```

6. Formatting and Indentation Matter

Consistent code is easier to read.

Bad:

```
class User{int age;String name;public User(int age,String name)
{this.age=age;this.name=name;}}
```

Good:

```
class User {
    private int age;
    private String name;

    public User(int age, String name) {
        this.age = age;
        this.name = name;
    }
}
```

7. Avoid Magic Numbers/Strings

Use named constants instead.

Bad:

```
if (user.age > 18) { // why 18?
    grantAccess();
}

double salary = hours * 15.50; // what is 15.50?
```

Good:

```
private static final int LEGAL_ADULT_AGE = 18;
private static final double HOURLY_RATE = 15.50;

if (user.age > LEGAL_ADULT_AGE) {
    grantAccess();
}

double salary = hours * HOURLY_RATE;
```

8. Prefer Composition Over Inheritance

Use composition to avoid tight coupling.

Bad:

```
class Employee extends Person { // fragile inheritance
    // duplicates logic
}
```

Good:

```
class Employee {
    private Person person; // composition

    public String getName() {
        return person.getName();
    }
}
```

Interview Questions on Clean Code

Q: How do you know if your code is clean?

> If a junior developer can understand it immediately without asking questions, it's clean. If it follows naming conventions, is well-structured, and requires minimal comments, it's clean.

Q: What's the most important aspect of clean code?

> Names. Good naming makes code self-documenting. If you have to explain a variable name, it's not clean.

Q: How do you handle technical debt?

> I refactor regularly, write tests to ensure safety, and document why shortcuts exist. Technical debt should be a conscious trade-off, not an accident.

SOLID Principles

Overview

SOLID is an acronym for five design principles that help create maintainable, scalable, and flexible software.

Principle	Meaning
S	Single Responsibility Principle
O	Open/Closed Principle
L	Liskov Substitution Principle
I	Interface Segregation Principle
D	Dependency Inversion Principle

S — Single Responsibility Principle (SRP)

Definition

A class should have only one reason to change. It should have only one job or responsibility.

Bad Example

```
class User {
    private String name;
    private String email;

    public void saveToDatabase() {} // Responsibility 1: persistence
    public void sendEmail(String message) {} // Responsibility 2: communication
    public void generateReport() {} // Responsibility 3: reporting
}
```

Good Example

```
class User {
    private String name;
    private String email;

    public String getName() { return name; }
    public String getEmail() { return email; }
}

class UserRepository {
    public void save(User user) { /* persistence logic */ }
}
```

```
class EmailService {
    public void sendEmail(String to, String message) { /* email logic */ }
}

class UserReportGenerator {
    public void generate(User user) { /* report logic */ }
}
```

☑ Each class has **one reason to change**: User data changes, persistence changes, email format changes, or report format changes.

O — Open/Closed Principle (OCP)

Definition

Classes should be **open for extension** but **closed for modification**. You should be able to add new features without modifying existing code.

Bad Example

```
class PaymentProcessor {
    public void process(String type, double amount) {
        if (type.equals("CARD")) {
            // process card
        } else if (type.equals("PAYPAL")) {
            // process paypal
        } else if (type.equals("CRYPTO")) {
            // process crypto
        }
        // Adding new payment type requires modifying this class
    }
}
```

Good Example

```
interface PaymentMethod {
    void process(double amount);
}

class CardPayment implements PaymentMethod {
    public void process(double amount) { /* card logic */ }
}

class PayPalPayment implements PaymentMethod {
    public void process(double amount) { /* paypal logic */ }
}

class PaymentProcessor {
```

```
    private PaymentMethod method;

    public PaymentProcessor(PaymentMethod method) {
        this.method = method;
    }

    public void process(double amount) {
        method.process(amount); // Extensible without modification
    }
}
```

☑ Adding a new payment type only requires creating a new class implementing `PaymentMethod`. No modification to existing code.

L — Liskov Substitution Principle (LSP)

Definition

Derived classes must be substitutable for their base classes without breaking the application. A subclass should not weaken the contract of the parent class.

Bad Example

```
class Bird {
    public void fly() { }
}

class Penguin extends Bird {
    @Override
    public void fly() {
        throw new UnsupportedOperationException("Penguins can't fly");
    }
}

// Problem: Code expecting Bird.fly() breaks when passed a Penguin
```

Good Example

```
interface Animal {
    void move();
}

class Bird implements Animal {
    public void move() { /* flying logic */ }
}

class Penguin implements Animal {
    public void move() { /* swimming logic */ }
```

```
}

// Now Penguin.move() doesn't violate the contract
```

☑ Subtypes must respect the contract of the parent type. If it can't, use composition or interfaces instead.

I — Interface Segregation Principle (ISP)

Definition

Clients should **not be forced to depend on interfaces they don't use**. Create small, focused interfaces instead of large, bloated ones.

Bad Example

```
interface Worker {
    void work();
    void eat();
    void sleep();
}

class Robot implements Worker {
    public void work() { /* ok */ }
    public void eat() { /* robot doesn't eat! */ }
    public void sleep() { /* robot doesn't sleep! */ }
}
```

Good Example

```
interface Workable {
    void work();
}

interface Eatable {
    void eat();
}

interface Sleepable {
    void sleep();
}

class Robot implements Workable {
    public void work() { }
}

class Human implements Workable, Eatable, Sleepable {
    public void work() { }
    public void eat() { }
```

```
    public void sleep() { }
}
```

☑ Interfaces are small and focused. Classes implement only what they need.

D — Dependency Inversion Principle (DIP)

Definition

1. High-level modules should not depend on low-level modules. Both should depend on abstractions.
2. Depend on abstractions, not concrete implementations.

Bad Example

```
class PaymentService {
    private StripePaymentGateway stripe = new StripePaymentGateway(); // Direct
dependency on concrete class

    public void processPayment(double amount) {
        stripe.charge(amount); // Tightly coupled
    }
}
```

Good Example

```
interface PaymentGateway {
    void charge(double amount);
}

class StripePaymentGateway implements PaymentGateway {
    public void charge(double amount) { }
}

class PaymentService {
    private PaymentGateway gateway;

    public PaymentService(PaymentGateway gateway) { // Injected dependency on
abstraction
        this.gateway = gateway;
    }

    public void processPayment(double amount) {
        gateway.charge(amount); // Loosely coupled
    }
}
```

☑ `PaymentService` depends on the abstraction `PaymentGateway`, not on `StripePaymentGateway`. Easy to swap implementations.

How SOLID Principles Work Together

```
User (Dependency Inversion)
  ↓
UserRepository (Interface - Dependency Inversion)
  ↑
DBRepository (Single Responsibility)
  ↑
Email Service (Single Responsibility)
```

Real-world Example: E-Commerce Order Processing

```java
// S: Each class has one job
class Order { /* order data */ }
class OrderRepository { /* persistence */ }
class PaymentGateway { /* payments */ }
class EmailNotifier { /* notifications */ }

// O: Can add new payment methods without changing OrderService
interface PaymentProcessor { void process(double amount); }
class CardProcessor implements PaymentProcessor { }
class CryptoProcessor implements PaymentProcessor { }

// L: Substitutable implementations
OrderService service = new OrderService(new CardProcessor());
// Later, swap it out
OrderService service = new OrderService(new CryptoProcessor());

// I: Small focused interfaces
interface Notifiable { void notify(String message); }
class EmailNotifier implements Notifiable { }

// D: Depend on abstractions
class OrderService {
    private PaymentProcessor processor;
    private Notifiable notifier;

    public OrderService(PaymentProcessor processor, Notifiable notifier) { }
}
```

Interview Questions on SOLID

Q: Why are SOLID principles important?

> They make code more maintainable, testable, and flexible. They reduce coupling and make future changes easier without breaking existing functionality.

Q: Give an example of violating SRP.

> A User class that handles user data, database persistence, email sending, and validation. It should have one reason to change: user data structure. Persistence, email, and validation should be in separate classes.

Q: How does DIP relate to Dependency Injection?

> DIP is the principle (depend on abstractions). DI is the pattern (injecting dependencies). DI is a practical way to implement DIP.

Q: Which SOLID principle is most violated?

> SRP. Classes often end up doing too much. Extract methods into separate classes when you see multiple reasons for change.

Q: How do you refactor code to follow SOLID?

> Start with SRP—identify classes doing too much and split them. Then apply D (use interfaces), L (ensure substitutability), and review if I or O apply. Use design patterns as helpers.

Q: Have you used SOLID in your code?

> Yes. I extract business logic into services (SRP), use interfaces for database access (DIP), design payment processors as strategies (O), keep interfaces focused (I), and ensure polymorphism is safe (L).